Answers to Algorithms, Programming and Logic Workbook

7 Algorithm design and problem solving

1 Three from:

- analysis – a process of investigation, leading to the specification of what a program is required to do
- design – uses the program specification from the analysis stage to show how the program should be developed
- coding – the writing of the program or suite of programs
- testing – the completed program is run with test data to make sure that all of the modules interact correctly, and the program as a whole works as intended in all circumstances

2 **Abstraction**

Stage Analysis

Use to identify the key elements required, discarding any unnecessary details and information that is not required

Decomposition

Stage Analysis

Use to break down a complex problem into smaller parts, repeating this until each part can be solved easily

Structure diagram

Stage Design

Use a hierarchical diagram that shows how a computer system can be repeatedly divided into sub-systems

3 a data

hardware

communications

b Three from:

- input
- process
- output
- storage

4 Structure diagrams are used to show top-down design in a diagrammatic form.

Flowcharts show diagrammatically the steps required to complete a task and the order that they are to be performed.

Pseudocode describes what an algorithm does by using English key words that are very similar to those used in a high-level programming language.

© David Watson and Helen Williams 2021

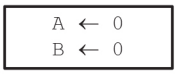
5 Structure diagrams are used to show the **top-down** design of a computer **program** in a **diagrammatic** form.

A structure diagram shows the design of a computer program in a **hierarchical** way with each **level** giving a more detailed **breakdown** of the **system** into **sub-systems**.

6 a



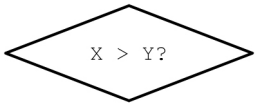
Terminator flowchart symbols are used at the beginning and end of each flowchart.



Process flowchart symbols are used to show actions.



Input and output symbols – the same flowchart symbol is used to show input of data and output of information.



Decision flowchart symbols are used to decide which action is to be taken next; these can be used for selection and repetition/iteration.

b Flowchart flow lines use arrows to show the direction of flow through the chart, which is usually, but not always, top to bottom and left to right.

7 Assignment: The single variable on the left of the ← is assigned the value of the expression on the right. For example:

```
Total ← Total + SubTotal
```

Conditional: When different actions are performed by an algorithm according to the values of the variables, conditional statements can be used to decide which action should be taken.

```
IF Age > 70
  THEN
    OUTPUT "Senior"
  ELSE
    OUTPUT "Adult or Junior"
ENDIF
```

Iterative: When some actions performed as part of an algorithm need repeating this is called iteration. Loop structures are used to perform the iteration.

```
FOR Counter ← 1 TO 10
    OUTPUT "*"
NEXT Counter
```


© David Watson and Helen Williams 2021

8 a Loop 1: `FOR .. TO .. NEXT:`

```
FOR Counter ← 0 TO 19
    OUTPUT Student[Counter]
NEXT Counter
```

Loop 2: `REPEAT .. UNTIL`

```
Counter ← 0
REPEAT
    OUTPUT Student[Counter]
    Counter ← Counter + 1
UNTIL Counter > 19
```

Loop 3: `WHILE .. DO .. ENDWHILE`

```
Counter ← 0
WHILE Counter < 20 DO
    Student[Counter]
    Counter ← Counter + 1
ENDWHILE
```

b i Conditional Statements 1: IF statements

```
OUTPUT "Please enter your name "
INPUT Name
IF Name = "Alice"
  THEN
    OUTPUT "Welcome Alice "
ENDIF
IF Name = "Abid"
  THEN
    OUTPUT "Welcome Abid "
ENDIF
IF Name = "Dinesh"
  THEN
    OUTPUT "Welcome Dinesh "
ENDIF
IF Name = "Daisy"
  THEN
    OUTPUT "Welcome Daisy "
ENDIF
IF Name = "Zak"
  THEN
    OUTPUT "Welcome Zak "
ENDIF
```


© David Watson and Helen Williams 2021

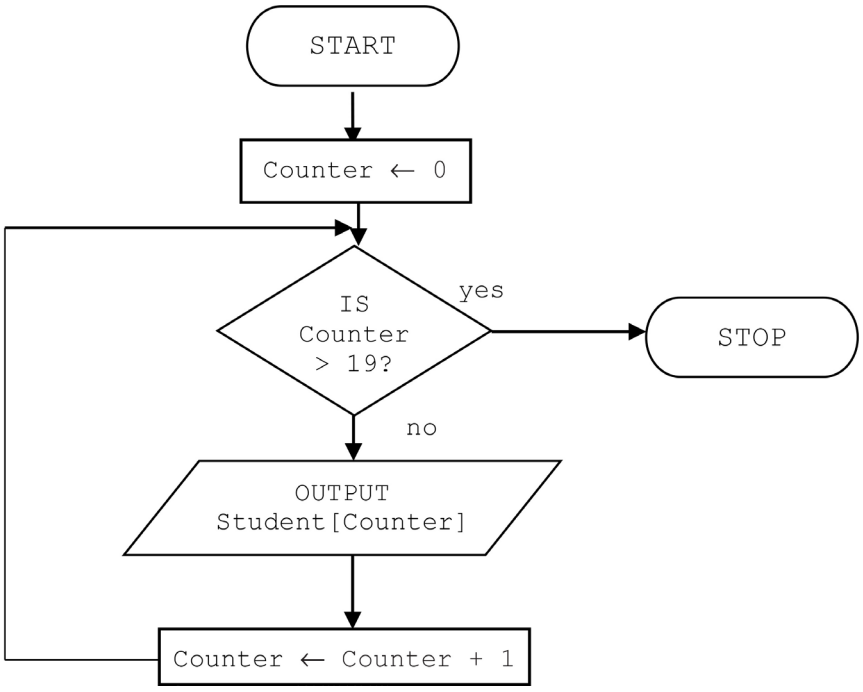
```
IF Name <> "Alice" AND Name <> "Abid" AND Name <> "Dinesh"
  AND Name <> "Daisy" <> AND Name <> "Zak"
  THEN
    OUTPUT "You are not welcome "
ENDIF
```

Conditional Statements 2: CASE statements

```
OUTPUT "Please enter your name "
INPUT Name
CASE OF Name
  "Alice"   : OUTPUT "Welcome Alice "
  "Abid"    : OUTPUT "Welcome Abid "
  "Dinesh"  : OUTPUT "Welcome Dinesh "
  "Daisy"   : OUTPUT "Welcome Daisy "
  "Zak"     : OUTPUT "Welcome Zak "
  OTHERWISE OUTPUT "You are not welcome "
ENDCASE
```

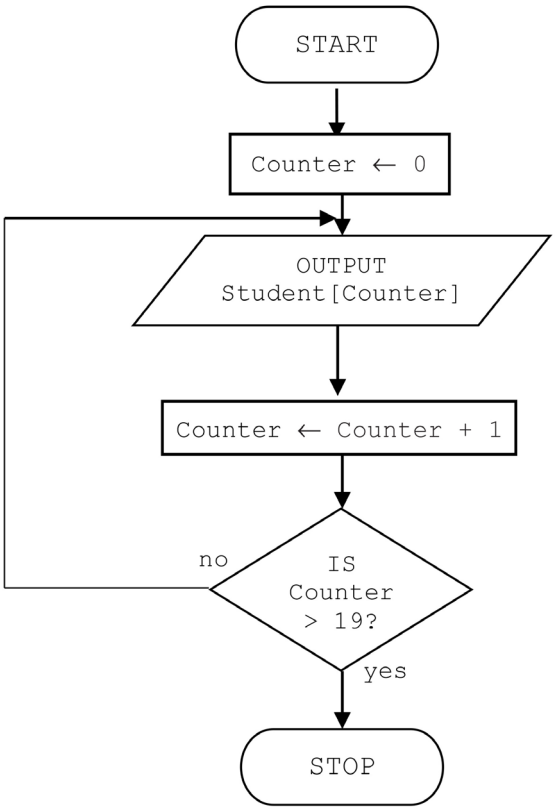
ii
- The most suitable conditional statement to use is `CASE` as the code is considerably shorter to write …
- … and clearer to read and understand.

c i Pre-condition loop



© David Watson and Helen Williams 2021

Post-condition loop



ii Pre-condition loop WHILE .. DO .. ENDWHILE

Post-condition loop REPEAT .. UNTIL

d Loop 1

```
FOR .. TO .. NEXT
```

Python

```
Student = ["Alice", "Jan", "Lim", "Ian", "Min", "Sue", "Tim",
"Dee", "Avril", "Ron", "Fred", "Idris", "May", "Susie", "Tony",
"Dom", "Ted", "Davy", "Chen", "Bao"]

Counter = 0
for Counter in range(20):
    print(Student[Counter])
print()
```

Visual Basic

```
Dim Student = New String() {"Alice", "Jan", "Lim", "Ian", "Min",
"Sue", "Tim", "Dee", "Avril", "Ron", "Fred", "Idris", "May",
"Susie", "Tony", "Dom", "Ted", "Davy", "Chen", "Bao"}

Dim Counter As Integer

'FOR .. TO .. NEXT
For Counter = 0 To 19
    Console.WriteLine(Student(Counter))
Next
Console.WriteLine()
```


© David Watson and Helen Williams 2021

Java

```
String[] Student ={"Alice", "Jan", "Lim", "Ian", "Min", "Sue",
"Tim", "Dee", "Avril", "Ron", "Fred", "Idris", "May", "Susie",
"Tony", "Dom", "Ted", "Davy", "Chen", "Bao"};
int Counter;

// FOR .. NEXT
    for (Counter = 0; Counter <= 19; Counter ++) {
        System.out.println(Student[Counter]);
    }
    System.out.println();
```

Loop 2

```
WHILE .. DO .. ENDWHILE
```

Python

```
Student = ["Alice", "Jan", "Lim", "Ian", "Min", "Sue", "Tim",
"Dee", "Avril", "Ron", "Fred", "Idris", "May", "Susie", "Tony",
"Dom", "Ted", "Davy", "Chen", "Bao"]

Counter = 0
while Counter < 20:
    print(Student[Counter])
    Counter = Counter + 1
print()
```

Visual Basic

```
Dim Student = New String() {"Alice", "Jan", "Lim", "Ian", "Min",
"Sue", "Tim", "Dee", "Avril", "Ron", "Fred", "Idris", "May",
"Susie", "Tony", "Dom", "Ted", "Davy", "Chen", "Bao"}

Dim Counter As Integer
'WHILE .. DO .. ENDWHILE
Counter = 0
While Counter < 20
    Console.WriteLine(Student(Counter))
    Counter = Counter + 1
End While
Console.WriteLine()
```


© David Watson and Helen Williams 2021

Java

```
String[] Student ={"Alice", "Jan", "Lim", "Ian", "Min", "Sue",
"Tim", "Dee", "Avril", "Ron", "Fred", "Idris", "May", "Susie",
"Tony", "Dom", "Ted", "Davy", "Chen", "Bao"};

int Counter;
// WHILE .. DO .. ENDWHILE
Counter = 0;
while (Counter <= 19){
    System.out.println(Student[Counter]);
    Counter ++;
}
System.out.println();
```

Loop 3

```
REPEAT .. UNTIL
```

Python Not available

Visual Basic

```
Dim Student = New String() {"Alice", "Jan", "Lim", "Ian", "Min",
"Sue", "Tim", "Dee", "Avril", "Ron", "Fred", "Idris", "May",
"Susie", "Tony", "Dom", "Ted", "Davy", "Chen", "Bao"}

Dim Counter As Integer
'REPEAT .. UNTIL
Counter = 0
Do
    Console.WriteLine(Student(Counter))
    Counter = Counter + 1
Loop Until Counter = 19
Console.WriteLine()
```

Java

```
String[] Student ={"Alice", "Jan", "Lim", "Ian", "Min", "Sue",
"Tim", "Dee", "Avril", "Ron", "Fred", "Idris", "May", "Susie",
"Tony", "Dom", "Ted", "Davy", "Chen", "Bao"};

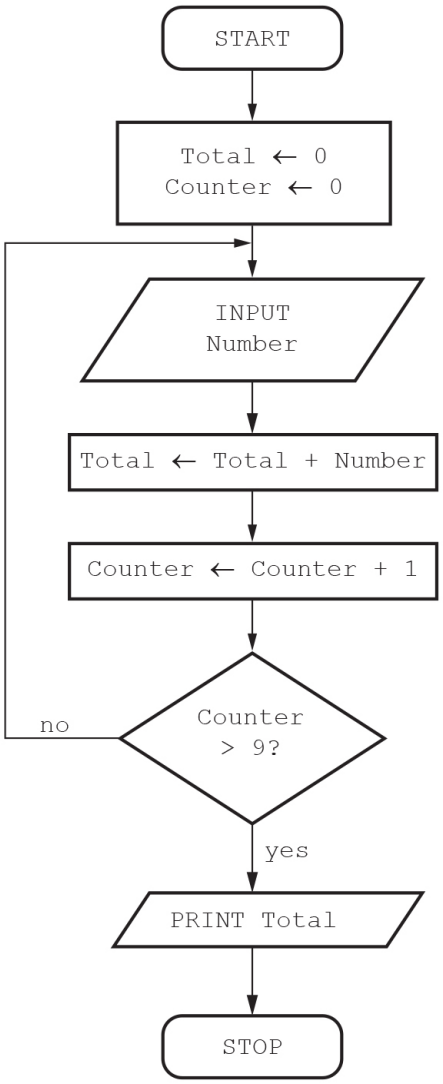
int Counter;
// REPEAT .. UNTIL
Counter = 0;
do {
    System.out.println(Student[Counter]);
    Counter ++;
}
while (Counter <= 19);
System.out.println();
```


© David Watson and Helen Williams 2021

9 a

```
DECLARE Number, Total : REAL
DECLARE Counter : INTEGER
Total ← 0
FOR Counter ← 1 TO 10
  OUTPUT "Please enter next Number"
  INPUT Number
  Total ← Total + Number
NEXT Counter
OUTPUT "The total of the 10 numbers input is ", Total
```

b



© David Watson and Helen Williams 2021

10 a input – the number of guesses allowed

input – a guess for a word

check – checks if a guess is correct

check – checks how many guesses are left

output – Message(s) to show if a guess is correct or not

b
- To play a word guessing game …
- … allowing the player to decide how many guesses to take.

c
```
// The code below represents the guessing algorithm.
// Assume that the word to guess, W, has already been
// declared and defined.
DECLARE N : INTEGER
DECLARE G : STRING
DECLARE Finish : BOOLEAN
Finish ← FALSE
OUTPUT "How many guesses would you like?"
INPUT N
REPEAT
    OUTPUT "Please enter your guess: "
    INPUT G
    IF G = W
      THEN
        OUTPUT "Well done you guessed the word!"
        Finish ← TRUE
      ELSE
        OUTPUT "You failed to guess the word!"
        N ← N - 1
    ENDIF
UNTIL Finish OR N = 0
```

11 Standard methods of solution are used in the design of **algorithms**. These include adding in a new value every time an action occurs, for example, awarding a mark to each student – this is called **totalling**. When divided by the number of times this occurs, this gives the **average**. Keeping a record of the number of times an action occurs is called **counting**. Finding the largest, **maximum**, and smallest, **minimum**, are also standard methods.

In order to look for an item in a list a **search** is used. The method you need to know for IGCSE Computer Science is to inspect each item in the **list** in turn to see if it is the one required. This is called a **linear search**.

To put a list in order a **sort** is used. The method you need to know for IGCSE Computer Science is called a **bubble sort**.

© David Watson and Helen Williams 2021

12 a
- Each element in a list is compared with the next element and swapped if the elements are in the wrong order …
- … starting from the first element and finishing with the next-to-last element.

b
```
First ← 1
Last ← 10
REPEAT
    Swap ← FALSE
    FOR Index ← First TO Last - 1
      IF Names[Index] > Names[Index + 1]
        THEN
          Temp ← Names[index]
          Names[index] ← Names[index + 1]
          Names[index + 1] ← Temp
          Swap ← TRUE
      ENDIF
    NEXT Index
    Last ← Last - 1
UNTIL (NOT Swap) OR (Last = 1)
```

13 a i Validation is appropriate to check that only reasonable data is accepted when data is entered.

ii Verification is appropriate to check that the data does not change as it is being entered.

b i range check

presence check

ii double entry

visual check

14 a
- Test data is a set of values used to determine whether a solution is working as it should …
- … the values are chosen to test a full range of operating conditions.

b i normal data – a set of test data used that the program would normally be expected to work with

abnormal/erroneous data – test data chosen to be rejected by the solution as not suitable, if the solution is working properly

extreme data – the largest and smallest values that normal data can take

ii 99 inside the boundary

100 outside the boundary

© David Watson and Helen Williams 2021

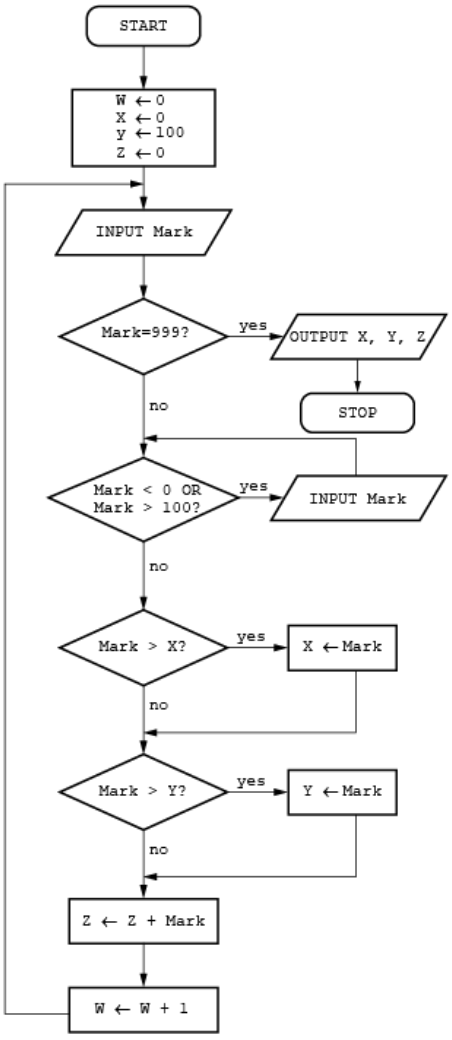
15 a

W	X	Y	Z	Mark	OUTPUT
0	0	100	0		
1	78	78	78	78	
2		34	112	34	
3		22	134	22	
				-4	
4	98		232	98	
5		16	248	16	
				734	
6			336	88	
				999	
					98 16 336

b

- selects the maximum and minimum values input between 0 and 100 …
- … calculates the total of these valid input values …
- … and outputs the maximum, minimum and the total.

c



© David Watson and Helen Williams 2021

d

```
NumberOfMarks ← 0
HighMark ← 0
LowMark ← 100
Total ← 0
REPEAT
    OUTPUT "Enter 999 to finish"
    OUTPUT "Marks must be in the range 0-100, enter next
      mark"
INPUT Mark
IF Mark <> 999
  THEN
    REPEAT
        IF Mark <0 OR Mark >100
          THEN
            OUTPUT "Your mark is out of range,
              please re-enter the mark"
            INPUT Mark
        ENDIF
    UNTIL Mark >=0 AND Mark <=100
    IF Mark > HighMark
      THEN
        HighMark ← Mark
    ENDIF
    IF Mark < LowMark
      THEN
        LowMark ← Mark
    ENDIF
  Total ← Total + Mark
  NumberOfMarks ← NumberOfMarks + 1
ENDIF
UNTIL Mark = 999
OUTPUT "The highest mark is ", HighMark
OUTPUT "The lowest mark is ", LowMark
OUTPUT "The total of all the marks is ", Total
```


© David Watson and Helen Williams 2021

e Example test data could be: Test data: −1, 0, 100, 101, −5, 21, 999

This has been chosen so that there is mixture of normal, abnormal, extreme and boundary data. In this case:

Normal data: **21**, **999** : 999 has been chosen to ensure that the method used to stop the program works and should be included in test data; but any other value in the range 1–99 could also be used.

Abnormal data: **−5** : but any other data larger than 101, less than −1, or of the wrong data type (e.g. twenty one) could be used

Extreme data: **0**, **100** : these are the only choice of extreme data for this example

Boundary data: **−1**, **0**, **100**, **101** : these are the only choice of boundary data for the main data range (although some students might also want to test the boundary for the 'quit' option, i.e. 998, 999, 1000).

Trace table:

NumberOf Marks	HighMark	Low Mark	Total	Mark	OUTPUT
0	0	100	0		Enter 999 to finish Marks must be in the range 0-100, enter next mark
				−1	Your mark is out of range, please re-enter the mark
1		0	0	0	Enter 999 to finish Marks must be in the range 0-100, enter next mark
2	100		100	100	Enter 999 to finish Marks must be in the range 0-100, enter next mark
				101	Your mark is out of range, please re-enter the mark
				−5	Your mark is out of range, please re-enter the mark
3			121	21	Enter 999 to finish Marks must be in the range 0-100, enter next mark
				999	The Highest mark is 100 The lowest mark is 0 The total of all the marks is 121

© David Watson and Helen Williams 2021